C-Grade Engine:A Rule-Driven Evaluator for C Programs

Manas Gogna (2611CS10)[1], Husain Kanchwala (2611CS04)[1], Jay Agrawal (2611AI09)[1], Adnan Hasan (2611MC09)[1]

[1]Department of Computer Science and Engineering, Indian Institute of Technology Patna, Bihta, Bihar, India

Abstract

Automated assessment of C programming submissions requires more than standard input/output comparison: a submission may produce correct output on sample inputs while still containing runtime faults, inefficient algorithms, poor structure, or memory-safety defects. This paper presents C-Grade Engine, a rule-driven framework for multi-dimensional assessment and profiling of C programming assignments. The system combines three major components: (i) a Gatekeeper and POSIX process sandbox that verifies source readiness and executes a bounded smoke test; (ii) nine specialist analyzers covering functional correctness, adversarial edge cases, compiler diagnostics, static quality, modularity, asymptotic complexity, runtime memory safety, readability, and submission similarity; and (iii) a report generator that aggregates normalized analyzer scores and presents evidence and counter-examples. Grading rules, metric weights, thresholds, compiler options, and execution limits are externalized in JSON and managed through a role-based web portal backed by SQLite. The platform also generates reference-based test cases using an instructor-provided oracle program, while allowing instructors to inspect and edit generated cases. We describe the system architecture, evaluation workflow, analyzer design, testing process, practical applications, and limitations.

1. Introduction

C is widely used in introductory and intermediate university courses because it exposes students to compilation, pointers, memory management, and algorithmic efficiency. Assessing C programs at scale is challenging because a submission can pass a small set of sample tests while retaining serious defects. Examples include signed integer overflow at boundary values, uninitialized variables, out-of-bounds array access, memory leaks, infinite loops, segmentation faults, and algorithms whose complexity exceeds the assignment requirement. Code may also be difficult to maintain when a large amount of logic is placed in `main()` with unclear identifiers and no meaningful decomposition.

Manual assessment is difficult to scale to classes of several hundred students. It is time-consuming and may lead to inconsistent grading or limited individualized feedback. Conventional online judges address an important part of this problem by comparing program output against expected output, but this alone does not evaluate memory safety, code structure, compiler diagnostics, or algorithmic efficiency.

C-Grade Engine addresses these gaps through a configurable assessment pipeline. It combines deterministic checks with selected static and dynamic analysis tools, and uses a local language model only for tasks that benefit from qualitative feedback. The system is designed around three principles:

- **Reproducible grading:** the same submission evaluated under the same rule configuration should produce the same result. Complexity profiling uses Valgrind Callgrind instruction counts rather than noisy wall-clock measurements. The local language-model configuration uses temperature zero and seed 42.
- **Instructor-editable rules:** analyzer selection, weights, thresholds, compiler options, complexity requirements, and sandbox limits are stored outside the analyzer implementation in JSON configuration and SQLite.
- **Safe execution and actionable feedback:** student binaries are executed under process, time, and memory limits. When a test fails, the system can report the input, expected output, actual output, and failure category.

The remainder of this paper presents the grading pipeline and sandbox (Section 2), the portal and database (Section 3), the nine analyzers (Section 4), the testing workflow (Section 6), practical applications and limitations (Section 7), team responsibilities (Section 8), and conclusions (Section 9).

2. Grading Pipeline, Sandbox, and Test Generation

C-Grade uses a staged evaluation process so that unsafe or invalid submissions are not passed blindly to execution-based analyzers. The pipeline consists of a Gatekeeper stage, specialist analysis, and report generation. The Gatekeeper decides which later checks are safe to run.

2.1. Gatekeeper and Sanity Sweep

When a student submits a C source file, `core/first_pass.py` performs three preliminary checks:

- **Static source sanity:** checks for an `int main` entry point, at least one recognized input function (`scanf`, `fscanf`, `fgets`, `getchar`, or `getline`), at least one output function (`printf`, `fprintf`, `puts`, or `putchar`), and a return statement in `main()`.
- **Sandboxed compilation:** compiles the submission using GCC with `-O2 -Wall -Wextra -std=c11`. Compilation failures are recorded with the compiler diagnostic.

- **Sample smoke test:** executes the compiled program using the assignment's sample input under configured time and memory limits (for example, 1 s and 64 MB).

If compilation and the smoke test succeed, enabled specialist analyzers are dispatched. If compilation fails, or the smoke test times out or crashes, execution-dependent analyzers are skipped. The system records a suspected infinite loop for a timeout or the observed crash signal, while static analyzers can still run. In particular, I/O tests, edge-case execution, complexity profiling, and sanitizer execution are skipped after an unsafe smoke-test result; compiler diagnostics, static analysis, function decomposition, readability review, and similarity analysis may still be performed.

2.2. Linux/POSIX Process Sandbox

The module `core/sandbox.py` isolates student binaries using POSIX process controls and resource limits. Each child process is started in a new session using `os.setsid`. This gives the parent process a process-group identifier that can be used with `os.killpg(..., SIGKILL)` to terminate the complete process tree if a program hangs or spawns child processes.

The sandbox applies a virtual address-space ceiling through `resource.RLIMIT_AS`, configured in bytes from the memory limit in megabytes. It also enforces kernel CPU limits through `resource.RLIMIT_CPU` and a Python wall-clock timeout. These controls bound programs that allocate excessive memory, loop indefinitely, recurse without termination, or block while waiting for input. Negative process return codes are decoded into student-facing diagnostics; examples include `-11` for `SIGSEGV`, `-8` for `SIGFPE`, and `-6` for `SIGABRT`.

2.3. Reference-Based Test-Case Generation

The `test_case_generator.py` module uses the instructor's reference implementation, `oracle.c`, to create expected-output test files. First, the oracle is compiled into an executable in the assignment's test directory. A local Ollama model (`qwen3.5:2b`) is then prompted with the problem statement, sample input, input type, and a code snippet from the oracle to generate varied inputs, including files such as `auto_01.in` and `auto_02.in`.

Because model-generated inputs may be malformed, the generator supplements them with deterministic edge-case templates based on the input format. These include zero, negative values, `INT_MAX`, `INT_MIN`, and one-element arrays. Each valid input is executed against the compiled oracle inside the sandbox, and its standard output is saved as the corresponding expected-output file. The generated cases can subsequently be reviewed and edited by an instructor through the portal.

3. Database, Authentication, and Web Portal

The command-line tools support development, but classroom deployment also requires persistent records, role-based access, and interfaces for managing assignments, reference solutions, and test cases. C-Grade provides these facilities through `database/database.py`, `database/auth.py`, `core/portal_rules.py`, and the Streamlit application in `app.py`.

3.1. Relational Database Architecture

The application uses SQLite (`portal.db`) with three principal tables:

- **Students:** stores the roll number as the primary key, student name, PBKDF2-hashed password, and creation timestamp. Instructors can import a class from a CSV file containing roll number and name, or manage individual student records.
- **Assignments:** stores the assignment slug, title, description, due date, time and memory limits, sample input, input type, expected complexity, oracle path, and `rules.json` configuration.
- **Submissions:** stores a unique submission identifier, student roll number, assignment slug, source-code path, timestamp, score, report, and complete JSON execution trace.

Deleting an assignment removes dependent submission records through database cascade behavior. The backend also removes the corresponding `problems/<problem-slug>` directory to avoid stale assignment files.

3.2. Role-Based Authentication and Session Persistence

C-Grade provides separate teacher and student login workflows. Teacher credentials are read from `local.env` using `TEACHER_USERNAME` and `TEACHER_PASSWORD`. Only authenticated teachers can manage assignments, test cases, student records, and class-wide submissions. Students log in using their roll number and password. Newly imported students receive the default password `123` and can change it through the Settings page; new passwords are hashed using PBKDF2 before storage.

To preserve login state across browser reloads, `database/auth.py` uses a token stored in URL query parameters. This addresses the loss of Streamlit session state that otherwise occurs when a page is reloaded.

3.3. Teacher and Student Workflows

The portal supports assignment creation and editing, including changes to expected complexity, sample input, deadlines, descriptions, and sandbox limits. Teachers upload or paste `oracle.c`; when it is updated, the system recompiles it and can regenerate expected outputs.

The test-case manager displays generated input/output pairs and allows instructors to edit, delete, or add cases. When an input is added or modified, the backend executes the compiled oracle to populate the expected output. This gives instructors a way to correct malformed model-generated inputs and remove invalid cases.

During assignment configuration, teachers select the analyzers to enable. `core/portal_rules.py` normalizes the selected analyzer weights so that they sum to 1.00 and computes a SHA-256 `rules_hash`. Students can view active assignments, upload a C file or paste source code, and submit once. Resubmission is locked after a submission has been recorded.

After evaluation, the application displays a plain-text, monospace report. Each metric's weighted contribution is calculated as

$$C_i = \text{round}(w_i s_i, 1), \tag{1}$$

where w_i is the normalized analyzer weight and s_i is its score. Feedback is wrapped to a fixed width to improve alignment and readability.

The teacher grading dashboard provides submissions across assignments, aggregate pass rates, similarity flags, saved reports, submitted source code, and the raw JSON evaluation history. This allows instructors to inspect the evidence behind each metric rather than relying only on the aggregate grade.

4. Specialist Analyzer Design

The nine specialist modules share a common evaluation contract. Each receives the student's source path, compiled binary path when applicable, rule parameters, and sandbox limits. It returns a normalized score in the range $[0, 1]$, structured evidence, and a concise feedback summary. The analyzers are summarized below.

4.1. Functional Correctness

The `analyzers/io_tests.py` module executes the student binary on every input/output pair in the assignment's test directory. Before comparison, it normalizes trailing whitespace at line ends and trailing blank lines to avoid failures caused only by formatting differences. The score is

$$S_{\text{IO}} = \frac{N_{\text{passed}}}{N_{\text{total}}}, \tag{2}$$

where N_{passed} and N_{total} denote the number of passed and total tests. Each failure is recorded with its input, expected output, actual output, and a note identifying a wrong answer, timeout, or runtime crash.

4.2. Adversarial Edge Cases

The `analyzers/edge_case_runner.py` module targets configured boundary categories, including negative values, zero, integer overflow, `INT_MAX` (2147483647), and `INT_MIN` (-2147483648). It compares the student's output and UndefinedBehaviorSanitizer status with the oracle. For example, signed overflow in an addition involving `INT_MAX` can be detected, and feedback can recommend using `long long` or checking bounds before performing the operation.

4.3. Compiler Diagnostics

The `analyzers/compiler_warnings.py` module runs GCC in syntax-only mode with `-Wall -Wextra -std=c11`. It parses warning messages from standard error, including warnings such as `-Wunused-variable`, `-Wunused-parameter`, `-Wsign-compare`, and `-Wimplicit-function-declaration`. Starting from a score of 1.0, it applies a configured fixed deduction (for example, 0.10) for each warning and records warning flags and source line numbers as evidence.

4.4. Static Analysis and Code Quality

The `analyzers/static_analysis.py` module combines three checks. First, Cppcheck is invoked with warning, style, performance, and portability checks to identify defects such as uninitialized variables, out-of-bounds indexing, resource leaks, and redundant conditions. Second, Lizard measures McCabe cyclomatic complexity and function length against configurable thresholds; the stated defaults are a maximum cyclomatic complexity of 8 and a maximum function length of 40 lines. Third, source-formatting heuristics check standard-header inclusion and flag code compressed into a single unformatted block.

4.5. Modularity and Function Decomposition

The `analyzers/function_split.py` module uses Lizard parsing to count defined functions. It compares this count with the configured `min_functions` requirement, typically two when helper functions are expected. A solution containing only `main()` receives a lower modularity score and feedback recommending that its core computation be moved into a helper function.

4.6. Asymptotic Complexity Estimation

The complexity pipeline consists of `input_spec.py`, `complexity_estimator.py`, and `complexity_fitting.py`. Wall-clock timing is avoided because operating-system scheduling, CPU frequency changes, and I/O buffering can introduce noise. Instead, the system generates synthetic inputs at increasing sizes (for example, $N = 16, 32, 64, 128, 256, 512$) and profiles the compiled binary using Valgrind Callgrind.

The estimator reads the executed instruction count, I_r, for each input size. Where the required Valgrind instrumentation is available, the implementation can isolate the target computation. The fitting module applies least-squares regression to candidate growth classes: $O(1)$, $O(\log N)$, $O(N)$, $O(N \log N)$, and $O(N^2)$. The inferred class is compared with the instructor's required complexity bound. The resulting raw (N, I_r) observations are retained for inspection.

4.7. Runtime Memory Safety

The `analyzers/sanitizer_run.py` module recompiles student code with GCC's AddressSanitizer and UndefinedBehaviorSanitizer using `-fsanitize=address,undefined -fno-omit-frame-pointer -g`. The instrumented binary is executed inside the sandbox. These sanitizers can detect heap, stack, and global buffer overflows, use-after-free errors, memory leaks, null-pointer dereferences, division by zero, and signed integer overflow. Diagnostics include stack traces and source locations when available.

4.8. Readability and Local LLM Review

The `analyzers/llm_review.py` module sends source code to a local Ollama HTTP endpoint running `qwen3.5:2b`. The request uses JSON output mode, temperature zero, and seed 42. The model scores readability out of 10 based on identifier

Table 1: Comparison Between Existing Judges and C-Grade

Metric	Traditional Judge	C-Grade Portal
Correctness	Black-box I/O only	Multi-dimensional (9 metrics)
Memory Safety	Ignored if exit is 0	ASan and UBSan checks
Complexity	Wall-clock time limit	Hardware-instruction fitting
Modularity Check	Not evaluated	Lizard AST helper validation
Compiler Warnings	Discarded or ignored	Strict parsing (`-Wextra`)
Plagiarism check	External batch scripts	Integrated AST/token Winnowing
Readability	None	Local deterministic LLM
Persistence Layer	Flat text files	SQLite relational schema

naming, comment quality, and consistent formatting. The analyzer extracts concise feedback.

If Ollama is unavailable or the model returns malformed JSON, the analyzer falls back to deterministic heuristics based on single-letter identifier ratios, comment density, and indentation consistency. Thus, the readability component can still return a result without a successful model response.

4.9. Originality and Similarity Detection

The `analyzers/similarity_flag.py` module compares submissions within an assignment. It removes comments, string literals, and preprocessor lines, then tokenizes the remaining C source. Variable and function identifiers are replaced with a canonical `ID` token and constants with `NUM`, reducing the effect of renaming and whitespace changes.

The analyzer forms overlapping token k-grams and applies Winnowing to select deterministic hash fingerprints. It then computes Jaccard similarity between the current submission and each other submission:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}, \tag{3}$$

where A and B are the fingerprint sets. A pair is flagged when its similarity meets or exceeds the configured threshold, which defaults to 0.85. The report identifies the matching submission and similarity percentage. A flag is intended to support instructor review, not to establish plagiarism automatically.

4.10. Comparison with Traditional Automated Judges

Table 1 summarizes the principal differences between a traditional output-based judge and the C-Grade Portal. The comparison highlights the additional analysis dimensions and supporting infrastructure integrated into C-Grade.

5. Fibonacci Assignment: End-to-End Workflow Demonstration

This section demonstrates the C-Grade workflow using a Fibonacci-number assignment. The teacher supplies a reference implementation (`oracle.c`) and configures the assignment rules. The system generates input/output test cases from the oracle. A student then submits a C source file, which passes through the Gatekeeper and, when safe to execute, the enabled specialist analyzers. The final report combines the metric scores and supporting feedback.

5.1. Workflow Overview

The workflow in Fig. 1 shows the path from assignment creation to result review. The teacher creates the assignment and saves the reference solution and grading rules. Test cases are generated using the oracle. Independently, the student submits a C source file. Both the generated tests and student submission feed into the Gatekeeper. If compilation and the smoke test pass, the enabled evaluation modules execute and their results are combined through weighted scoring. If either Gatekeeper check fails, the system generates a partial report. Both paths save a result for teacher and student review.

5.2. Teacher's Reference Solution (`oracle.c`)

The teacher's reference implementation defines the expected behavior of the assignment. The input is a non-negative integer n, and the output is the n-th Fibonacci number, where $F_0 = 0$ and $F_1 = 1$. The example restricts n to $0 \leq n \leq 92$, so the result fits in a signed 64-bit integer.

Listing 1: Teacher reference implementation for the Fibonacci assignment.

```c
#include <stdio.h>

long long fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;

    long long a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
       long long next = a + b;
       a = b;
       b = next;
    }
    return b;
}

int main(void) {
    int n;
    if (scanf("%d", &n) != 1) return 1;
    if (n < 0 || n > 92) return 1;

    printf("%lld\n", fibonacci(n));
    return 0;
}
```

The reference implementation uses an iterative recurrence, requiring $O(n)$ time and $O(1)$ auxiliary space. The test-case generator compiles this program and runs it on each generated input to produce the expected output.

5.3. Student Submission (`fibonacci.c`)

The following student submission is an illustrative implementation created for this demonstration. It uses a helper function and descriptive identifiers while implementing the same iterative recurrence as the oracle.

Listing 2: Illustrative student submission for the Fibonacci assignment.

```c
#include <stdio.h>

long long calculate_fibonacci(int number) {
    long long previous = 0;
    long long current = 1;

    if (number == 0) {
       return 0;
    }

    for (int index = 1; index < number; index++) {
```

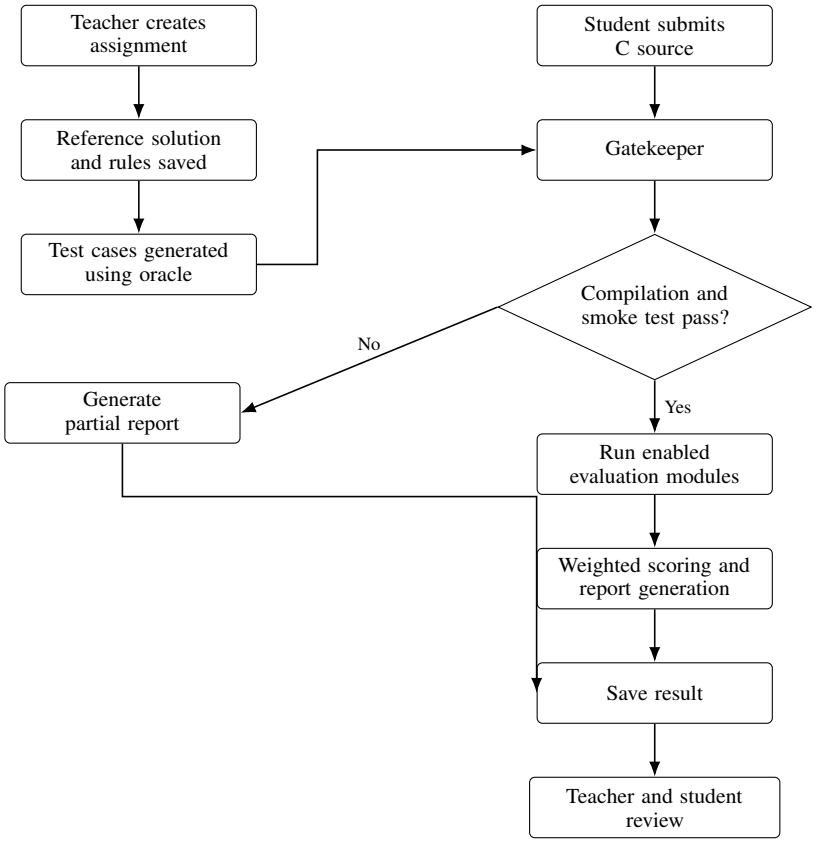


Fig. 1: C-Grade Engine workflow for a teacher-created Fibonacci assignment and student submission.

```
        long long next_value = previous + current;
        previous = current;
        current = next_value;
    }

    return current;
}

int main(void) {
    int number;

    if (scanf("%d", &number) != 1) {
        return 1;
    }

    if (number < 0 || number > 92) {
        return 1;
    }

    printf("%lld\n", calculate_fibonacci(number));
    return 0;
}
```

For the stated input range, the student implementation is functionally equivalent to the reference solution. It uses one helper function, iterative computation, and constant auxiliary storage.

5.4. Test Cases and Expected Outputs

Table 2 gives representative inputs and expected Fibonacci outputs. These examples illustrate the test-case format; the five successful tests reported by the project may contain additional or different values depending on the assignment's generated test files.

Table 2: Representative Fibonacci Test Cases

Input n	**Expected output** F_n
0	0
1	1
2	1
10	55
20	6765

5.5. Sample Evaluation Report

The following table reorganizes the supplied console report for `fibonacci.c`. The reported overall grade is 98.5/100. All metrics receive full marks except readability, which receives 8.5/10.

The supplied report states that all five I/O tests and all five edge cases passed. It reports no runtime-safety failures, static-analysis issues, compiler warnings, or errors. The complexity estimate matches the expected $O(N)$, and the readability analyzer assigns 8.5/10. The report also recommends that the student continue the same quality of work.

Two details in the supplied console output should be interpreted carefully. First, the modularity feedback names `has_target_pair`, whereas the illustrative Fibonacci student program in Listing 2 defines `calculate_fibonacci`. This name is reproduced from the supplied report and should be checked against the actual submitted source and saved evaluation trace. Second, the similarity result is reported as zero against `None`; this indicates that no comparison candidate

Table 3: Sample C-Grade Engine Evaluation Report for `fibonacci.c`

Metric	Weight	Score	Contribution	Reported feedback
Correctness	20	20/20	20.0	Passed all 5/5 I/O test cases.
Logical	10	10/10	10.0	Handled all 5/5 edge cases cleanly.
Memory safety	10	10/10	10.0	Passed all runtime safety checks.
Complexity	10	10/10	10.0	Estimated $O(N)$; expected $O(N)$.
Static analysis	10	10/10	10.0	No static analysis issues detected.
Compiler warnings	10	10/10	10.0	No compiler warnings or errors detected.
Modularity	10	10/10	10.0	Good modularity; report detected `main()` and one helper function, `has_target_pair`.
Plagiarism check	10	10/10	10.0	Similarity 0.0 (Struct: 0.0, Lit: 0.0) against `None`.
Readability	10	8.5/10	8.5	Highly readable code with excellent naming and formatting.
	Overall grade		**98.5/100**	

was named in the output and should not be treated as conclusive evidence of originality.

5.6. Demonstration Summary

The Fibonacci example connects the teacher's assignment configuration and oracle, oracle-based test generation, student source submission, Gatekeeper validation, specialist analyzer execution, and weighted report generation. It demonstrates how C-Grade combines functional correctness with static, dynamic, structural, complexity, readability, and similarity checks while preserving metric-level feedback for review.

6. Testing and Quality Assurance

The project integrates GCC, POSIX resource controls, Callgrind, sanitizers, Lizard, Cppcheck, SQLite, Streamlit, and a local language model. Testing was therefore organized in progressive stages, beginning with individual analyzers and proceeding to end-to-end workflow validation.

6.1. Individual Analyzer Testing

The command-line harness `testing/test_tool.py` allows a developer to run one analyzer against a selected C source file and rule configuration. Representative invocations are:

```
python testing/test_tool.py --source
submissions/add.c
  --analyzer io_tests --rules
rules/add_rules.json
python testing/test_tool.py --source
submissions/add.c
  --analyzer edge_case_runner --rules
rules/add_rules.json
python testing/test_tool.py --source
submissions/add.c
  --analyzer sanitizer_run --rules
rules/add_rules.json
```

Each analyzer was tested using correct and deliberately faulty C programs. The test inputs included compiler warnings, monolithic `main()` functions, memory leaks, infinite loops, `INT_MAX` overflow, and nested loops with $O(N^2)$ behavior. The team inspected terminal JSON output to verify scores, evidence dictionaries, and feedback.

6.2. End-to-End Workflow Validation

After isolated analyzer checks, `testing/ui.py`, working with `testing/agent.py`, provided an intermediate Streamlit interface for validating the full workflow. It exercised assignment creation, automatic test generation, student submission, Gatekeeper smoke testing, analyzer execution, and report generation.

The interface displayed generated test cases and raw analyzer results. This helped identify integration issues, including incorrect oracle paths passed to the edge-case analyzer and partial reports produced when the Gatekeeper smoke test timed out. The final `app.py` portal was then used to validate the integrated classroom workflow.

7. Practical Applications and Limitations

7.1. Practical Applications

C-Grade is intended for settings where output correctness alone is insufficient.

- **Large introductory programming courses:** in a course with approximately 400 students, identical rule configurations can be applied to all submissions. The sandbox contains crashes and infinite loops, while per-metric scores and counter-examples provide students with more specific feedback than a pass/fail result.
- **First-round technical hiring:** for C programming assessments, the platform can evaluate functional correctness alongside sanitizer results, instruction-based complexity estimates, and modularity. Its JSON evaluation history provides an audit trail for engineering reviewers.
- **Plagiarism triage:** for a class of 300 submissions, pairwise manual comparison would involve up to 44,850 unique pairs. Token normalization and Winnowing can narrow instructor attention to pairs whose structural similarity crosses the configured threshold.

7.2. System Limitations and Mitigations

The system has several limitations that affect the interpretation of its results.

1) **Small local language model:** `qwen3.5:2b` can produce malformed JSON or invalid test inputs, particularly for structured multi-line formats. For example, a generated array input may declare $N = 5$ but provide only three values,

or may contain explanatory prose. The project mitigates this with strict parsing, deterministic readability fallbacks, and an instructor-facing test-case manager that supports inspection, editing, deletion, and manual additions.

2) **Finite test suites:** passing all generated tests does not prove correctness. A maximum-element program that initializes its result to zero may pass tests containing positive values but fail on an all-negative array. The edge-case analyzer and instructor-defined tests reduce this risk, but finite black-box testing cannot establish mathematical correctness.
3) **Incorrect reference implementation:** the oracle determines expected outputs. A bug in `oracle.c`, such as 32-bit overflow in an integer-sum problem, can cause correct student output to be marked wrong. Instructors can update the oracle and inspect or edit test input/output pairs through the portal, but the reference implementation must still be validated.
4) **Complexity misclassification:** least-squares fitting over a modest range of input sizes can confuse growth classes when constant factors, parsing, or library I/O dominate the instruction count. The system retains raw (N, I_r) measurements so instructors can inspect the evidence rather than treating the fitted class as definitive.
5) **Similarity false positives:** short boilerplate programs may be structurally similar even when written independently. For example, simple input/output programs can exceed a 0.90 similarity score after identifier normalization. Similarity is therefore reported as a review indicator, not an automatic penalty; instructors may also disable the analyzer for assignments dominated by boilerplate.

8. Team Responsibilities

The four-member team divided development into module ownership areas and collaborated on integration testing. Table 4 summarizes the reported responsibilities and principal contributions.

9. Conclusion and Future Work

This paper presented C-Grade Engine, an end-to-end, rule-driven assessment and profiling platform for C programming assignments. The system combines a POSIX process sandbox, a Gatekeeper stage, nine specialist analyzers, and a persistent SQLite-backed Streamlit portal. It extends conventional output-based grading with checks for edge-case behavior, compiler diagnostics, static code quality, modularity, instruction-based complexity, runtime memory safety, readability, and submission similarity.

The staged testing workflow uses `testing/test_tool.py` for individual analyzer validation, `testing/ui.py` for intermediate end-to-end testing, and `app.py` for the integrated portal. Instructor controls for editing test cases and updating `oracle.c` are important safeguards, particularly because the local language model may generate malformed inputs. The platform aims to make assessment more consistent and transparent by retaining normalized scores together with the evidence used to compute them.

Future work includes rootless Docker and seccomp-based sandboxing for public multi-tenant hosting, support for larger 7B–14B local coder models when suitable GPU hardware is available, multi-file C projects and Makefile-based grading, and visual cohort analytics in the Teacher Portal.

References

[1] Free Software Foundation, "GCC, the GNU Compiler Collection—Options to Request or Suppress Warnings and Instrumentation Options." [Online]. Available: https://gcc.gnu.org/onlinedocs/

[2] N. Nethercote and J. Seward, "Valgrind: A framework for heavyweight dynamic binary instrumentation," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2007. [Online]. Available: https://valgrind.org/docs/manual/cl-manual.html

[3] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," in *Proc. USENIX Annual Technical Conf. (USENIX ATC)*, 2012.

[4] S. Schleimer, D. S. Wilkerson, and A. Aiken, "Winnowing: Local algorithms for document fingerprinting," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2003.

[5] D. Marjamäki, "Cppcheck: A tool for static C/C++ code analysis." [Online]. Available: https://cppcheck.sourceforge.io/

[6] T. Yin, "Lizard: An extensible cyclomatic complexity analyzer." [Online]. Available: https://github.com/terryyin/lizard

[7] Ollama Team, "Ollama local model runner (qwen3.5:2b)." [Online]. Available: https://ollama.com/

[8] Streamlit Inc., "Streamlit framework documentation." [Online]. Available: https://docs.streamlit.io/

Table 4: Team Workload Distribution and Modular Ownership

Member	Role	Assigned Modules	Key Contributions
Manas Gogna (2611CS10)	Sandbox, rules, and plagiarism lead	`core/first_pass.py`, `core/sandbox.py`, `core/portal_rules.py`, `testing/agent.py`, `analyzers/llm_review.py`, `analyzers/similarity_flag.py`	Gatekeeper, POSIX sandbox, dynamic rule normalization, SHA-256 rule hashing, LLM review integration, and Winnowing-based similarity engine.
Husain Kanchwala (2611CS04)	Dynamic analysis, sanitizers, and test harness lead	`analyzers/io_tests.py`, `analyzers/edge_case_runner.py`, `analyzers/sanitizer_run.py`, `test_case_generator.py`, `testing/ui.py`, `testing/test_tool.py`	Black-box I/O runner, boundary edge-case analyzer, ASan/UBSan module, oracle-based test generator, and CLI test harness.
Jay Agrawal (2611AI09)	Static analysis, web application, and authentication lead	`app.py`, `database/database.py`, `database/auth.py`, `analyzers/compiler_warning.py`, `analyzers/function_split.py`, `core/reporter.py`	Streamlit interface, SQLite schema, authentication, GCC warning parser, function decomposition analyzer, and plain-text reporter.
Adnan Hasan (2611MC09)	Complexity profiling and static analysis lead	`analyzers/complexity_est.py`, `analyzers/complexity_fit.py`, `analyzers/input_spec.py`, `analyzers/static_analysis.py`	Callgrind complexity profiler, curve-fitting engine, input specification generator, and Cppcheck/Lizard integration.